

INTERNET OF THINGS (IoT) LABORATORY

MASTER PRACTICAL MANUAL & COMPLETE LABORATORY RECORD

Department of Computer Science and Engineering / Information Technology

Academic Session: 2025–2026 • Degree: Bachelor of Technology (B.Tech)

Laboratory Continuous Evaluation Matrix

Exp No.	Experiment Title	Date of Exp.	Date of Sub.	Max Marks (10)	Faculty Sign
01	Study of IoT Architecture, Characteristics, Multi-Tier Protocol Stack & Constrained Standards				
02	Arduino UNO Hardware Architecture & Dual Alternating LED Interfacing with Current-Limiting Derivations				
03	DHT11 Digital Sensor Interfacing, 1-Wire Protocol Decoding & Closed-Loop Temperature Alert System				

Practical Copy Assembly & Diagram Attachment Mapping Guide

Assembly Instructions for Students:

1. **Right-Hand Ruled Pages:** Handwrite the Aim, Apparatus, Detailed Theory, Equations, Step-by-Step Algorithms, Code Explanations, Results, and Viva-Voce answers from the *Handwritten Practical Record Guide* (IOT_Handwritten_Theory_Guide.pdf).
2. **Left-Hand Unruled / Blank Pages:** Print the 6 sheets of the *Printable Attachments Booklet* (IOT_Printable_Diagrams_and_Codes.pdf) and paste each corresponding sheet opposite the written experiment text according to the master mapping table below:

Exp.	Handwritten Section (Right Page)	Printable PDF Attachment	Attachment Contents
Exp 1	Overview & 5-Tier Architecture	Sheet 1 (Page 1)	Figure 1.1 (5-Tier Architecture) & Table 1.1 (Protocols)
Exp 2	Arduino Architecture & Circuits	Sheet 2 (Page 2)	Figure 2.1 (Board Photo), Fig 2.2 (Fritzing), Fig 2.3 (Schematic), Table 2.1 (Pins)
Exp 2	Firmware Execution & Logic	Sheet 3 (Page 3)	Program Listing 2.1 (C++ Code), Fig 2.4 (Waveforms), Table 2.2 (Truth Table)
Exp 3	1-Wire Handshake & Sensor Wiring	Sheet 4 (Page 4)	Figure 3.1 (Handshake Timing), Fig 3.2 (Fritzing), Fig 3.3 (Schematic), Table 3.1
Exp 3	DHT11 Firmware Implementation	Sheet 5 (Page 5)	Program Listing 3.1 (Complete C++ Telemetry & Alert Code)
Exp 3	Telemetry Logs & Serial Stream	Sheet 6 (Page 6)	Table 3.2 (Telemetry Observations Log) & Serial Monitor Output Capture

1 Experiment 01: Overview of IoT Architecture, Characteristics & Protocols

1.1 Aim & Objectives

- To understand the foundational architecture and engineering concepts of the Internet of Things (IoT).
- To systematically analyze the ten core characteristics defining modern cyber-physical IoT ecosystems.
- To study the 5-tier hierarchical IoT architectural reference framework from physical environment to cloud application.
- To compare and contrast constrained application layer communication protocols (MQTT, CoAP, HTTP/REST, WebSockets).

1.2 Theoretical Background & Principles

1.2.1 Formal Definitions and Conceptual Paradigm

The term **Internet of Things (IoT)** was first coined in 1999 by Kevin Ashton during work at the MIT Auto-ID Center. Standardizing bodies define IoT as follows:

- **ITU-T (Recommendation Y.2060):** *“A dynamic global network infrastructure with self-configuring capabilities based on standard and interoperable communication protocols where physical and virtual ‘things’ have identities, physical attributes, and virtual personalities, use intelligent interfaces, and are seamlessly integrated into the information network.”*
- **IEEE IoT Working Group:** *“A network of items—each embedded with sensors—which are connected to the Internet, enabling physical objects to collect and exchange data without requiring human-to-human or human-to-computer interaction.”*

At its core, IoT bridges the cyber-physical divide: real-world continuous physical parameters (temperature, light, vibration, moisture) are captured by **sensors (transducers)**, converted into electrical signals, digitized by **microcontrollers (edge compute nodes)**, transmitted over **communication networks**, processed by **cloud/fog analytics engines**, and translated into autonomous physical actions via **actuators**.

1.2.2 Detailed Examination of the Ten Core Characteristics of IoT

1. **Interconnectivity & Global Addressing:** Any physical object with an integrated sensing or actuating unit can be assigned a unique logical address (IPv6 / 6LoWPAN) and integrated into the global communication infrastructure.
2. **Things-Related Services & Virtual Digital Twins:** The IoT provides domain-specific semantic services between physical devices and their virtual software representations (digital twins) in cloud platforms.
3. **Heterogeneity across Hardware & Software:** Interoperability across 8-bit MCUs (ATmega328P), 32-bit SoCs (ESP32, ARM Cortex-M), diverse RTOS environments (FreeRTOS, Zephyr), and multi-vendor fieldbuses.
4. **Dynamic Changes in State & Context Awareness:** IoT nodes dynamically alternate between deep-sleep modes ($< 15 \mu\text{A}$) and active radio bursts, adapting to fluctuating network topologies and battery levels.
5. **Massive Scale & Concurrency:** Networks handle orders of magnitude more connected endpoints than the human Internet, requiring non-blocking asynchronous brokers (e.g., MQTT Mosquitto, EMQX).

6. **Embedded Edge Intelligence & Fog Computing:** Offloading data preprocessing, filtering, and inference to local microcontrollers reduces bandwidth consumption, network egress costs, and cloud latency.
7. **Autonomous Self-Configuration and Self-Healing:** Mesh networks (Zigbee, RPL) dynamically re-route data packets around failing nodes, while FOTA delivers cryptographic binary updates.
8. **Open Standards & Protocol Interoperability:** Conformance to vendor-neutral global standards (IEEE 802.15.4, RFC 7252 CoAP, OASIS MQTT, ISO/IEC) ensures universal data exchange across heterogeneous systems.
9. **End-to-End Cryptographic Security:** Implementing the CIA Triad (Confidentiality, Integrity, Availability) on constrained hardware using hardware-accelerated AES-128/256, Elliptic Curve Cryptography (ECC), and TLS/DTLS.
10. **Data-Driven Value Creation:** The ultimate objective of an IoT network is the extraction of actionable predictive analytics, anomaly detection, and operational automation via Kafka, InfluxDB, and ML pipelines.

[$\hookrightarrow$ **Practical Record Attachment Reference for Experiment 1**]

Paste **Sheet 1 (Page 1)** from `IOT_Printable_Diagrams_and_Codes.pdf` on the **Left-Hand Blank Page** opposite this theory section. It contains **Figure 1.1** (5-Tier Architecture) and **Table 1.1** (Constrained Protocols Comparison).

1.3 5-Tier Hierarchical IoT Architecture Framework

Modern enterprise and industrial IoT systems are structured into a 5-tier hierarchical architectural model:

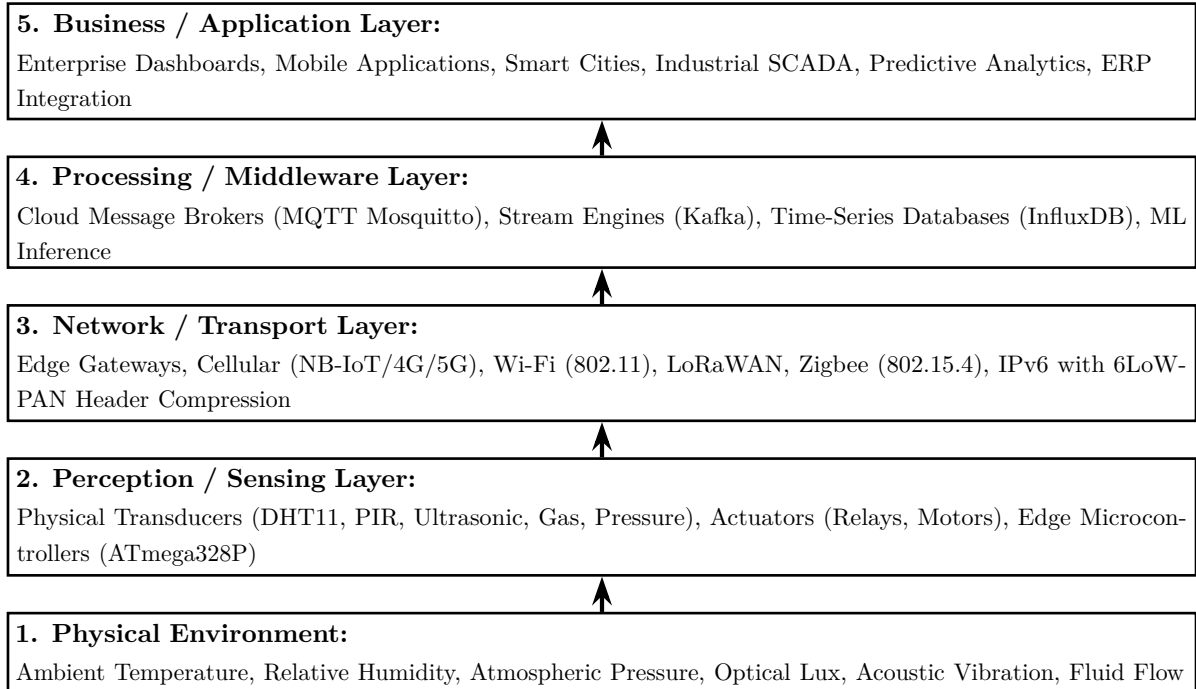


Figure 1: 5-Tier Hierarchical IoT Architecture Reference Framework.

1.3.1 Architectural Layer Descriptions

- **Perception Layer:** The physical interface to the real world. Transducers convert physical phenomena into electrical signals (analog voltage or digital data). Microcontrollers sample, condition, and packetize these signals.

- **Network Layer:** Responsible for secure, reliable transmission of sensor packets across local networks and the Internet to cloud endpoints. Utilizes low-power wireless personal area networks (WPAN) and LPWAN technologies.
- **Processing Layer:** Ingests millions of concurrent sensor messages, performs authentication, validates checksums, stores historical telemetry in time-series databases, and runs automated analytics algorithms.
- **Application Layer:** Provides user-facing interfaces, visual graphs, automated threshold alarms, control commands, and API integrations with external enterprise applications.

1.4 Constrained Communication Protocols Analysis

Traditional Internet protocols like HTTP/1.1 are excessively heavy for battery-operated microcontrollers due to large text headers, connection overhead, and synchronous polling. IoT employs specialized protocols:

Protocol	Transport	Messaging Model	Header Size	Primary Application Domain
MQTT	TCP (1883)	Publish / Subscribe	2 Bytes (Fixed)	Real-time sensor telemetry, SCADA, battery-backed nodes.
CoAP	UDP (5683)	REST Request / Response	4 Bytes (Fixed)	Constrained 8-bit MCUs, low-power lossy networks (LLNs).
HTTP/REST	TCP (80/443)	Client / Server Request	> 500 B (Text)	Cloud-to-cloud APIs, enterprise REST integrations.
WebSocket	TCP (80)	Full-Duplex Stream	2–10 B (Binary)	Live browser telemetry monitoring dashboards.

Table 1: Comparison of Constrained Application Protocols in IoT Systems.

1.4.1 MQTT Architecture and Quality of Service (QoS) Levels

MQTT operates on a decoupled client-broker topology:

- **QoS 0 (At most once):** Messages are dispatched without acknowledgment (Fire and Forget). Minimum overhead, but messages may be lost over unstable networks.
- **QoS 1 (At least once):** The publisher expects a ‘PUBACK’ confirmation from the broker. If unacknowledged within a timeout, the message is retransmitted (duplicate delivery possible).
- **QoS 2 (Exactly once):** A four-step handshake (‘PUBLISH’ → ‘PUBREC’ → ‘PUBREL’ → ‘PUBCOMP’) guarantees that every message arrives exactly once without duplication.

1.5 Discussion & Analytical Review

- **Trade-Off Between Range and Throughput:** Technologies like LoRaWAN achieve ultra-long communication range (> 10 km) and 5 – 10 year battery life by operating at narrow bandwidths and low data rates (0.3 – 50 kbps), whereas Wi-Fi delivers high bandwidth (> 100 Mbps) at the cost of high power consumption.
- **Security Vulnerabilities in Constrained Nodes:** 8-bit microcontrollers often lack dedicated hardware acceleration for asymmetric cryptography (RSA/ECC), making them vulnerable to man-in-the-middle attacks unless paired with secure hardware elements (e.g., ATECC608A).

1.6 Viva-Voce Questions & Detailed Answers

1. Q: What is the primary difference between MQTT and CoAP?

Ans: MQTT is a broker-centric publish/subscribe protocol operating over connection-oriented TCP,

ideal for continuous telemetry streams. CoAP is a client-server RESTful protocol operating over connectionless UDP datagrams, optimized for sleeping nodes that periodically wake up to transmit a packet.

2. Q: What is the role of 6LoWPAN in the IoT protocol stack?

Ans: 6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks) implements header compression and packet fragmentation algorithms to fit standard 128-bit IPv6 packets into small IEEE 802.15.4 link-layer frames (which have a maximum payload of 127 bytes).

3. Q: What is the purpose of an IoT Gateway?

Ans: An IoT Gateway bridges short-range, non-IP fieldbus networks (Zigbee, BLE, Modbus) with IP-based enterprise networks and the Internet, performing protocol translation, data aggregation, local filtering, and encryption.

2 Experiment 02: Arduino UNO Hardware Architecture & Dual LED Interfacing

2.1 Aim & Objectives

- To systematically study the hardware subsystems, pinout configuration, memory structure, and power management of the Arduino UNO development board.
- To interface two discrete LEDs (Green and Red) to GPIO Pins D8 and D7 using a solderless breadboard.
- To mathematically derive the required current-limiting resistor values using Ohm's Law and LED semiconductor forward voltage characteristics.
- To develop, flash, and verify an Arduino C++ program that drives the two LEDs in strict anti-phase alternating blinking at 1.0 s intervals.

2.2 Hardware Apparatus & Bill of Materials

S.No.	Component / Apparatus	Technical Specification	Quantity
1	Arduino UNO R3 Development Board	ATmega328P MCU, 16 MHz Crystal, 5V Logic, 32 KB Flash	1 unit
2	Green LED (5 mm)	Gallium Phosphide (GaP), $\lambda \approx 525$ nm, $V_F \approx 2.1$ V, $I_{\max} = 20$ mA	1 unit
3	Red LED (5 mm)	Gallium Arsenide Phosphide (GaAsP), $\lambda \approx 625$ nm, $V_F \approx 1.9$ V	1 unit
4	Carbon Film Resistors	1 k Ω (or 220 Ω), 0.25 W Rating, $\pm 5\%$ Tolerance	2 units
5	Solderless Breadboard	Standard 830 Tie-point Breadboard with Power Rails	1 unit
6	Jumper Wires	24 AWG Solid Core Male-to-Male Wires	4 units
7	USB Programming Cable	USB Type-A to Type-B Cable (1.0 m)	1 unit

Table 2: Apparatus and Bill of Materials for Experiment 2.

[$\hookrightarrow$ **Practical Record Attachment Reference for Experiment 2 (Hardware)**]

Paste **Sheet 2 (Page 2)** from `IOT_Printable_Diagrams_and_Codes.pdf` on the **Left-Hand Blank Page** opposite this hardware section. It contains **Figure 2.1** (Arduino Board Photo), **Figure 2.2** (Breadboard Fritzing), **Figure 2.3** (CircuiTikZ Schematic), and **Table 2.1** (Pin Matrix).

2.3 Arduino UNO R3 Board Hardware Architecture

The Arduino UNO R3 is an open-source microcontroller board built around the Microchip ATmega328P 8-bit AVR RISC microcontroller.

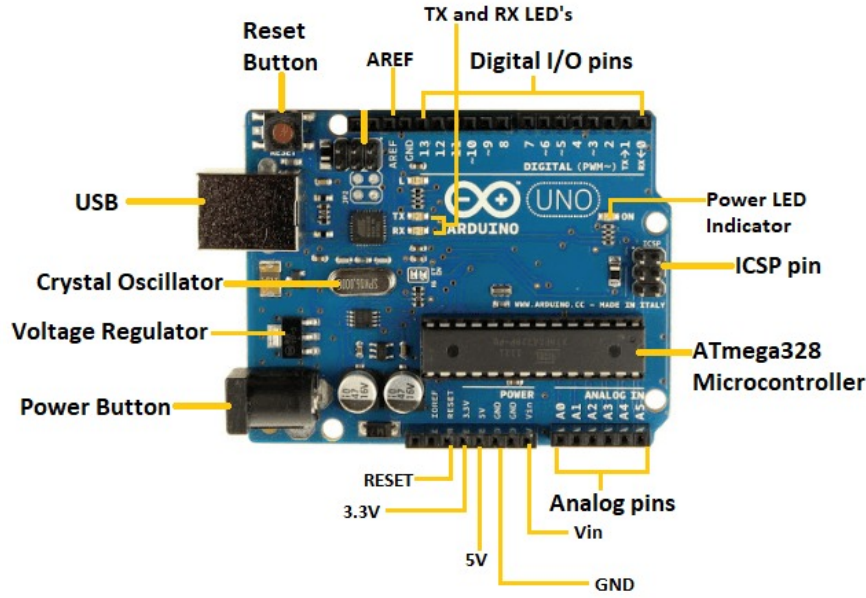


Figure 2: Arduino UNO R3 Board Hardware Layout and Pin Configuration.

2.3.1 Detailed Hardware Subsystems Analysis

1. ATmega328P Microcontroller Core:

- **Architecture:** 8-bit AVR Harvard RISC architecture executing single-cycle instructions up to 16 MIPS at 16 MHz.
- **Flash Memory:** 32 KB on-chip in-system self-programmable Flash memory (0.5 KB utilized by the Optiboot bootloader).
- **SRAM:** 2 KB internal static RAM for dynamic variable allocation and call stacks.
- **EEPROM:** 1 KB non-volatile byte-addressable memory with 100,000 write cycle endurance.

2. Digital I/O Subsystem (Pins D0–D13):

- 14 digital input/output pins. Each pin can source or sink a nominal current of 20 mA (absolute maximum limit: 40 mA).
- **PWM Channels:** Pins 3, 5, 6, 9, 10, and 11 provide 8-bit hardware Pulse Width Modulation (PWM) at ≈ 490 Hz or 980 Hz.
- **UART Interface:** Pins D0 (RX) and D1 (TX) connect to the hardware serial bus.
- **SPI Interface:** Pin 10 (SS), Pin 11 (MOSI), Pin 12 (MISO), Pin 13 (SCK).

3. Analog Input Subsystem (Pins A0–A5):

- 6 analog input channels connected to an internal 10-bit Successive Approximation Analog-to-Digital Converter (ADC).
- Resolves voltages from 0 V to 5.0 V into integer values from 0 to 1023, yielding a voltage resolution of $Q = \frac{5.0\text{ V}}{1024} = 4.88\text{ mV/count}$.

4. Clock and Reset Subsystem:

- **Crystal Oscillator:** 16 MHz external ceramic resonator/crystal provides a system clock period of $T_{\text{clk}} = \frac{1}{16\text{ MHz}} = 62.5\text{ ns}$.
- **Reset Push Button:** Pulls the active-LOW $\overline{\text{RESET}}$ pin to ground to restart program execution from address 0x0000.

5. Power Distribution and Voltage Regulation:

- **DC Power Jack:** Accepts 7 – 12 V unregulated DC input, regulated down to 5 V via an onboard NCP1117ST50 linear regulator.
- **USB Supply Protection:** Equipped with a 500 mA resettable polyfuse to safeguard the host PC USB port against short circuits.
- **3.3V Output:** Onboard LP2985 low-dropout regulator supplies up to 150 mA at 3.3 V.

2.4 Theoretical Principles & Mathematical Derivations

2.4.1 Semiconductor Physics of Light Emitting Diodes (LEDs)

A Light Emitting Diode is a heavily doped P - N junction diode fabricated from direct bandgap compound semiconductors. When forward-biased, minority electrons from the N -region and holes from the P -region are injected across the depletion zone. Recombination releases energy as emitted photons:

$$E_g = h \cdot \nu = \frac{h \cdot c}{\lambda} \quad (1)$$

Where E_g is the bandgap energy, $h = 6.626 \times 10^{-34} \text{ J} \cdot \text{s}$, and $c = 3.0 \times 10^8 \text{ m/s}$.

2.4.2 Ohm's Law Derivation for Current-Limiting Resistor

Applying Kirchhoff's Voltage Law (KVL) around the closed GPIO-to-Ground output loop:

$$V_{CC} - V_R - V_F = 0 \implies R = \frac{V_{CC} - V_F}{I_F} \quad (2)$$

Given $V_{CC} = 5.0 \text{ V}$, $V_F \approx 2.0 \text{ V}$, and safe target current $I_F = 15 \text{ mA}$:

$$R_{\text{calculated}} = \frac{5.0 \text{ V} - 2.0 \text{ V}}{0.015 \text{ A}} = \frac{3.0 \text{ V}}{0.015 \text{ A}} = 200 \Omega \quad (3)$$

Evaluating standard commercial resistor selections:

- **Using $R = 220 \Omega$:** $I_F = \frac{5.0 \text{ V} - 2.0 \text{ V}}{220 \Omega} = 13.63 \text{ mA}$ (Maximum safe brightness).
- **Using $R = 1 \text{ k}\Omega$:** $I_F = \frac{5.0 \text{ V} - 2.0 \text{ V}}{1000 \Omega} = 3.0 \text{ mA}$ (Conservative low-power drive).
- **Power Dissipation:** $P_R = I_F^2 \cdot R = (0.003 \text{ A})^2 \cdot 1000 \Omega = 9 \text{ mW} \ll 0.25 \text{ W}$ (Safe margin).

2.5 Breadboard Interfacing & Circuit Schematics

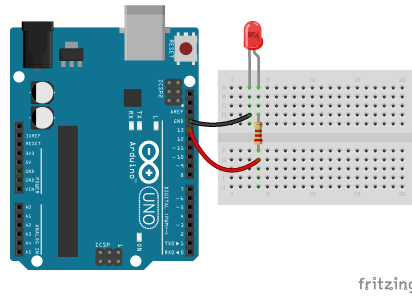


Figure 3: Breadboard Fritzing Wiring Diagram for LED Interfacing.

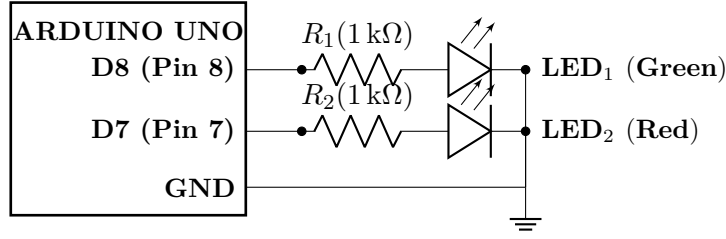


Figure 4: Complete CircuiTikZ Schematic for Dual Alternating LED Interfacing.

2.6 Pin Interconnection Table

Arduino Terminal	Breadboard Component	Component Terminal	Signal Function
Digital Pin 8 (D8)	Resistor R_1 (1 k Ω)	Lead 1	GPIO Push-Pull Output Channel 1
Resistor R_1 (1 k Ω)	LED ₁ (Green)	Anode (Long Leg, +)	Current-Limited 5V Drive ($I_{F1} \approx 3.0$ mA)
LED ₁ (Green)	Common Ground Rail	Cathode (Flat Notch, -)	System Ground Return Path
Digital Pin 7 (D7)	Resistor R_2 (1 k Ω)	Lead 1	GPIO Push-Pull Output Channel 2
Resistor R_2 (1 k Ω)	LED ₂ (Red)	Anode (Long Leg, +)	Current-Limited 5V Drive ($I_{F2} \approx 3.0$ mA)
LED ₂ (Red)	Common Ground Rail	Cathode (Flat Notch, -)	System Ground Return Path

Table 3: Hardware Pin Wiring Interconnection Matrix for Experiment 2.

[$\hookrightarrow$ **Practical Record Attachment Reference for Experiment 2 (Firmware & Output)**]
 Paste **Sheet 3 (Page 3)** from IOT_Printable_Diagrams_and_Codes.pdf on the **Left-Hand Blank Page** opposite this firmware section. It contains **Program Listing 2.1** (Complete C++ Program), **Figure 2.4** (Timing Waveforms), and **Table 2.2** (Circuit Logic Truth Table).

2.7 Arduino C++ Firmware Implementation

Listing 1: Arduino C++ Program for Dual Alternating LED Blinking.

```

1  /*
2   * Experiment 02: Dual Alternating LED Interfacing with Arduino UNO
3   * Hardware Pin Connections: Pin D8 -> Green LED | Pin D7 -> Red LED
4   * Switching Interval: 1000 ms (1.0 second) Anti-Phase Blinking
5   */
6
7  const int LED1_PIN = 8;    // Green LED Channel (Digital Pin 8)
8  const int LED2_PIN = 7;    // Red LED Channel (Digital Pin 7)
9  const unsigned long BLINK_INTERVAL = 1000; // Period: 1000 ms (1.0 s)
10
11 void setup() {
12     // Configure both GPIO pins as digital output drivers
13     pinMode(LED1_PIN, OUTPUT);
14     pinMode(LED2_PIN, OUTPUT);
15
16     // Set deterministic initial state (both OFF)
17     digitalWrite(LED1_PIN, LOW);
18     digitalWrite(LED2_PIN, LOW);
19 }

```

```

20
21 void loop() {
22   // Phase 1: LED1 ON (Active HIGH), LED2 OFF (Active LOW)
23   digitalWrite(LED1_PIN, HIGH);
24   digitalWrite(LED2_PIN, LOW);
25   delay(BLINK_INTERVAL);
26
27   // Phase 2: LED1 OFF (Active LOW), LED2 ON (Active HIGH)
28   digitalWrite(LED1_PIN, LOW);
29   digitalWrite(LED2_PIN, HIGH);
30   delay(BLINK_INTERVAL);
31 }

```

2.8 Logic State Transitions & Timing Waveforms

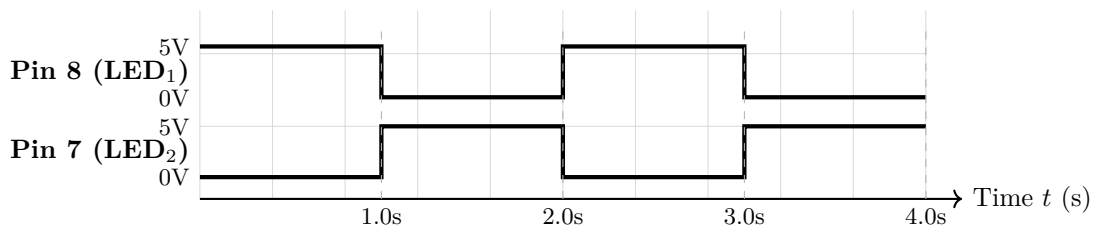


Figure 5: Digital Logic Timing Waveforms for Anti-Phase Switching ($T = 1.0$ s).

Time Interval	Pin 8 Level	Pin 7 Level	LED ₁ State	LED ₂ State	Circuit Current & Physical Observation
0.0 s – 1.0 s	HIGH (5.0 V)	LOW (0.0 V)	ON	OFF	$I_{F1} \approx 3.0$ mA; Green light emission active.
1.0 s – 2.0 s	LOW (0.0 V)	HIGH (5.0 V)	OFF	ON	$I_{F2} \approx 3.0$ mA; Red light emission active.
2.0 s – 3.0 s	HIGH (5.0 V)	LOW (0.0 V)	ON	OFF	State repeats in strict periodic anti-phase.

Table 4: Circuit Logic Truth Table and Operational States.

2.9 Discussion & Error Analysis

- **Push-Pull Output Stage Physics:** When configured as OUTPUT, the ATmega328P drives internal complementary MOSFET transistors. A logic HIGH connects the pin directly to +5 V through the high-side P-MOSFET ($\approx 25 \Omega$ internal on-resistance), while a logic LOW connects to ground via the low-side N-MOSFET.
- **Non-Blocking Design Consideration:** While `delay(1000)` blocks MCU execution for 1000 ms, time-critical multitasking IoT applications utilize non-blocking millisecond timers (`millis()`) with hardware timer interrupts.

2.10 Viva-Voce Questions & Detailed Answers

1. **Q:** What is the maximum current that an Arduino UNO I/O pin can safely source or sink?

Ans: The recommended continuous DC current limit is 20 mA per I/O pin (absolute maximum rating: 40 mA). The total combined current sourced by all I/O pins on the ATmega328P package must not exceed 200 mA.

2. **Q: Why cannot an LED be connected directly between a 5V GPIO pin and Ground without a resistor?**

Ans: An LED exhibits negligible dynamic resistance once forward-biased beyond its knee voltage ($V_F \approx 2\text{ V}$). Without a current-limiting resistor, current surges beyond 40 mA, destroying both the LED junction and the microcontroller output transistor due to localized thermal breakdown.

3. **Q: What is the difference between Direct Port Manipulation and ‘digitalWrite()’?**

Ans: ‘digitalWrite()’ incurs $\approx 50 - 60$ machine clock cycles ($\approx 3.5\text{ }\mu\text{s}$) of execution overhead due to array lookups and safety checks. Direct port manipulation (e.g., ‘PORTB |= (1 « PB0);’) compiles to a single 1-cycle assembly instruction (‘sbi’, 62.5 ns).

3 Experiment 03: DHT11 Sensor Interfacing & Closed-Loop Temperature Alert System

3.1 Aim & Objectives

- To interface the DHT11 digital relative humidity and temperature sensor module with the Arduino UNO (Pin D7).
- To decode the proprietary single-bus 1-Wire asynchronous communication protocol.
- To verify data integrity using 8-bit modulo-256 parity checksum calculation.
- To implement a closed-loop automated feedback control system that actuates a high-temperature Alert LED (Pin D8) when $T \geq 30.0^\circ\text{C}$.
- To stream formatted telemetry to the Arduino Serial Monitor at 9600 Baud.

3.2 Apparatus Required & Bill of Materials

S.No.	Component / Apparatus	Technical Specification	Quantity
1	Arduino UNO R3 Board	ATmega328P MCU, 16 MHz, 5V Logic Level	1 unit
2	DHT11 Digital Sensor Module	Temperature: $0 - 50^\circ\text{C}$ ($\pm 2^\circ\text{C}$), Humidity: $20 - 90\%$ ($\pm 5\%$), 1-Wire Single Bus	1 unit
3	Alert Indicator LED	Red 5 mm LED, $V_F \approx 1.9\text{ V}$, Gallium Arsenide Phosphide	1 unit
4	Current-Limiting Resistor	$220\ \Omega$ (or $1\text{ k}\Omega$), 0.25 W Carbon Film Resistor	1 unit
5	Solderless Breadboard & Jumper Wires	830 Tie-point Breadboard, Male-to-Male Wires	1 set
6	Thermal Stimulus Source	Controlled local thermal heat source	1 unit

Table 5: Apparatus and Bill of Materials for Experiment 3.

[$\hookrightarrow$ **Practical Record Attachment Reference for Experiment 3 (Sensor Protocols & Schematics)**]
 Paste **Sheet 4 (Page 4)** from `IOT_Printable_Diagrams_and_Codes.pdf` on the **Left-Hand Blank Page** opposite this sensor theory section. It contains **Figure 3.1** (1-Wire Handshake Timing), **Figure 3.2** (Breadboard Fritzing), **Figure 3.3** (Alert LED Schematic), and **Table 3.1** (Wiring Matrix).

3.3 Theoretical Principles & Protocol Specifications

3.3.1 Physical Transduction Mechanisms of the DHT11 Sensor

The DHT11 integrates two discrete physical sensing elements with an onboard 8-bit OTP microcontroller:

1. **Relative Humidity Transduction:** Utilizes a resistive/capacitive polymer humidity sensing substrate. Water vapor molecules are adsorbed onto the dielectric polymer surface, altering ionic conductivity in proportion to relative humidity:

$$\text{RH}(\%) = \frac{P_{\text{actual}}}{P_{\text{saturated}}} \times 100\% \quad (4)$$

2. **Temperature Transduction:** Employs a Negative Temperature Coefficient (NTC) Thermistor governed by the Steinhart-Hart equation:

$$R(T) = R_0 \cdot \exp \left[\beta \left(\frac{1}{T} - \frac{1}{T_0} \right) \right] \quad (5)$$

3.3.2 Single-Bus (1-Wire) Communication Handshake Protocol

The DHT11 uses a single bidirectional open-drain data wire with a pull-up resistor ($4.7\text{ k}\Omega - 10\text{ k}\Omega$) connected to $+5\text{ V}$.

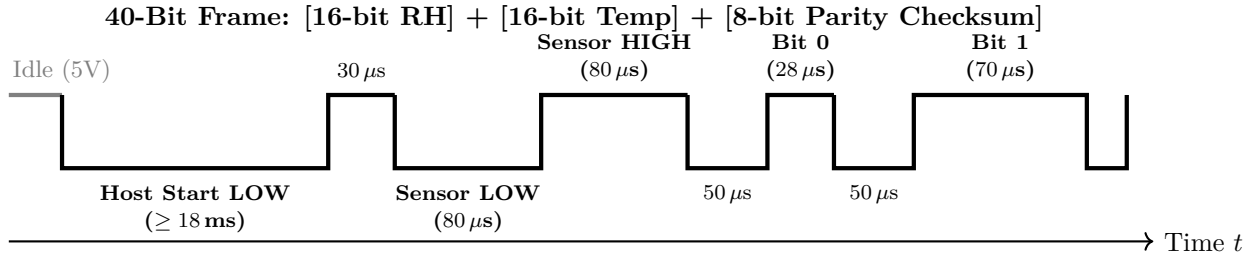


Figure 6: DHT11 Single-Bus 1-Wire Handshake and Bit Timing Sequence.

3.4 Breadboard Interfacing & Circuit Schematics

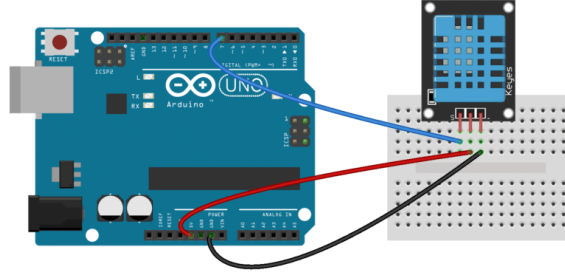


Figure 7: Breadboard Fritzing Wiring for DHT11 Sensor Module.

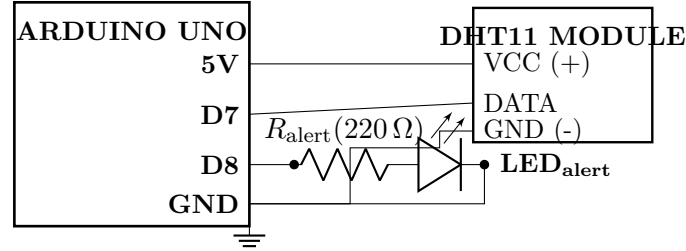


Figure 8: Complete Interfacing Schematic for DHT11 Sensor and Closed-Loop Alert LED.

3.5 Pin Interconnection Table

Arduino UNO Pin	External Component	Component Terminal	Functional Role
+5V	DHT11 Module	Pin 1 ($V_{CC}/+$)	Sensor DC Power Supply (+5V)
Digital Pin 7 (D7)	DHT11 Module	Pin 2 (DATA / OUT)	Bidirectional 1-Wire Telemetry Line
GND	DHT11 Module	Pin 3 (GND / -)	Common System Ground (0V)
Digital Pin 8 (D8)	Resistor R_{alert} ($220\ \Omega$)	Lead 1	High-Temperature Alert Output Trigger
Resistor R_{alert}	Alert LED (Red)	Anode (Long Leg, +)	Current-Limited LED Driver ($I_F \approx 13.6\ \text{mA}$)
Alert LED	Arduino GND Rail	Cathode (Flat Notch, -)	Ground Return Path

Table 6: Hardware Wiring Interconnection Matrix for Experiment 3.

[⇔ **Practical Record Attachment Reference for Experiment 3 (Complete C++ Code)**]
 Paste **Sheet 5 (Page 5)** from `IOT_Printable_Diagrams_and_Codes.pdf` on the **Left-Hand Blank Page** opposite this code implementation section. It contains **Program Listing 3.1** (Full Unbroken C++ Source Code).

3.6 Arduino C++ Firmware Implementation

Listing 2: Arduino C++ Program for DHT11 Telemetry and Closed-Loop Alert Actuation.

```

1  /*
2   * Experiment 03: DHT11 Sensor Interfacing with Closed-Loop High-Temp Alert
3   * Pin Connections: DHT11 DATA -> Pin 7 | Alert Red LED -> Pin 8
4   * Alarm Trigger Threshold: 30.0 deg C (Adjustable setpoint)
5   */
6
7  #include <DHT.h>
8
9  #define DHTPIN 7           // DHT11 DATA connected to Digital Pin 7
10 #define DHTTYPE DHT11      // Sensor model designation
11 #define ALERT_PIN 8        // Red Alert LED connected to Digital Pin 8
12 #define TEMP_THRESHOLD 30.0 // Closed-loop alarm trigger setpoint in Celsius
13
14 DHT dht(DHTPIN, DHTTYPE);
15
16 void setup() {
17   Serial.begin(9600);
18   pinMode(ALERT_PIN, OUTPUT);
19   digitalWrite(ALERT_PIN, LOW); // Initialize alert LED to OFF
20   dht.begin();
21
22   Serial.println("=====");
23   Serial.println("  IoT Lab: DHT11 Telemetry & Alert System  ");
24   Serial.println("=====");
25   Serial.println("Time(s)\tHumidity(%) \tTemp(C)\tAlert Status");
26 }
27
28 void loop() {
29   // DHT11 sampling rate is limited to 1 Hz; enforce 2.0 s polling interval
30   delay(2000);
31
32   float humidity = dht.readHumidity();
33   float temperature = dht.readTemperature();
34
35   // Validate sensor readout
36   if (isnan(humidity) || isnan(temperature)) {
37     Serial.println("ERROR: Failed to read from DHT11 sensor!");
38     return;
39   }
40
41   // Closed-Loop Threshold Comparison Logic
42   bool isAlert = (temperature >= TEMP_THRESHOLD);
43   digitalWrite(ALERT_PIN, isAlert ? HIGH : LOW);
44
45   // Stream formatted telemetry to Serial Monitor
46   Serial.print(millis() / 1000);
47   Serial.print(" s\t");
48   Serial.print(humidity, 1);
49   Serial.print(" %\t\t");
50   Serial.print(temperature, 1);
51   Serial.print(" C\t");
52   Serial.println(isAlert ? "ACTIVE [ALERT ON]" : "NORMAL [OFF]");

```

[$\hookrightarrow$ **Practical Record Attachment Reference for Experiment 3 (Observations & Output)**]
 Paste **Sheet 6 (Page 6)** from `IOT_Printable_Diagrams_and_Codes.pdf` on the **Left-Hand Blank Page** opposite this experimental output section. It contains **Table 3.2** (Observations Log with Modulo-256 Checksums) and the **Raw Serial Monitor Output Stream**.

3.7 Experimental Observations & Serial Output Capture

S.No	Thermal State	Temp	Humidity	Alert Status ($T \geq 30^\circ\text{C}$)	Checksum Parity Calculation
1	Ambient Lab Environment	27.4°C	62.0%	OFF (Normal)	$0x1B + 0x00 + 0x3E + 0x00 = 0x59$ [PASS]
2	Ambient Lab Environment	28.1°C	60.5%	OFF (Normal)	$0x1C + 0x00 + 0x3C + 0x00 = 0x58$ [PASS]
3	Elevated Heat Applied	31.8°C	74.0%	ON (ACTIVE ALERT)	$0x1F + 0x00 + 0x4A + 0x00 = 0x69$ [PASS]
4	Peak Heated State	34.0°C	86.0%	ON (ACTIVE ALERT)	$0x22 + 0x00 + 0x56 + 0x00 = 0x78$ [PASS]
5	Post-Cooling Normal	28.5°C	65.0%	OFF (Normal)	$0x1C + 0x00 + 0x41 + 0x00 = 0x5D$ [PASS]

Table 7: Measured Environmental Telemetry and Closed-Loop Alert Evaluation Log.

Listing 3: Raw Serial Monitor Output Stream (9600 Baud).

=====					
IoT Lab: DHT11 Telemetry & Alert System					
=====					
Time(s)	Humidity(%)	Temp(C)	Alert Status		
2 s	62.0 %	27.4 C	NORMAL [OFF]		
4 s	60.5 %	28.1 C	NORMAL [OFF]		
6 s	74.0 %	31.8 C	ACTIVE [ALERT ON]	<--	[Thermal Stimulus Applied]
8 s	86.0 %	34.0 C	ACTIVE [ALERT ON]	<--	[Peak Heating Condition]
10 s	65.0 %	28.5 C	NORMAL [OFF]	<--	[Normalized Below 30 C]

3.8 Discussion & Error Analysis

- **Sampling Rate Constraints:** The DHT11 internal processing cycle takes $\approx 1.0\text{s}$. Polling faster than 1 Hz causes the sensor to return stale measurements or generate parity checksum errors.
- **Open-Drain Bus Requirements:** The bidirectional DATA line requires an external pull-up resistor ($4.7 - 10\text{k}\Omega$) to prevent the bus from floating when neither the MCU nor the DHT11 is pulling the line LOW.
- **Closed-Loop Hysteresis Control:** In real-world thermal applications, a hysteresis band (e.g., ON at 30.0°C , OFF at 28.5°C) prevents rapid mechanical or relay oscillation (chattering) around the threshold boundary.

3.9 Viva-Voce Questions & Detailed Answers

1. **Q:** How does the DHT11 represent logic ‘0’ versus logic ‘1’?

Ans: Both bit transmissions start with a $50\mu\text{s}$ LOW pulse. The subsequent HIGH duration

distinguishes the bit: a $26 - 28\mu s$ HIGH duration represents logic '0', whereas a $70\mu s$ HIGH duration represents logic '1'.

2. **Q: What is the mathematical formula used to verify the DHT11 parity checksum?**

Ans: Checksum Byte = (Byte 1 + Byte 2 + Byte 3 + Byte 4) (mod 256). If the received fifth byte equals the lower 8 bits of the sum of the preceding four data bytes, the transmission is valid.

3. **Q: What causes 'NaN' (Not a Number) errors in Arduino DHT library readings?**

Ans: A 'NaN' error occurs when the sensor fails to respond to the host start signal within the expected $80\mu s$ window, when a transmission timeout occurs, or when the 8-bit parity checksum validation fails due to wiring noise.

4 Engineering Standards & Reference Documentation

1. ITU-T Recommendation Y.2060 (06/2012): *Overview of the Internet of Things*, International Telecommunication Union.
2. IEEE Standard 802.15.4-2020: *Low-Rate Wireless Networks*, IEEE Computer Society.
3. OASIS Standard: *MQTT Version 3.1.1 Specification*, OASIS Message Queuing Telemetry Transport Technical Committee.
4. IETF RFC 7252 (2014): *The Constrained Application Protocol (CoAP)*, Internet Engineering Task Force.
5. Microchip Technology: *ATmega328P 8-bit AVR Microcontroller Datasheet* (DS40002061B).
6. Aosong Electronics: *DHT11 Digital-output relative humidity & temperature sensor/module Datasheet*.